



YISHUN INNOVA JUNIOR COLLEGE
JC 2 MID YEAR PRACTICAL EXAM
Higher 2

CANDIDATE
NAME

CG

INDEX NO

COMPUTING

Paper 2 (Lab-based)

9569/02

26 May 2022

3 hours

100 Marks

Additional Materials: Removable storage device with the following files:

- `template.ipynb` (for Question 1, 2, 3 and 5)
- `COUNTRY1.TXT`, `COUNTRY2.TXT`
- `CLASS.TXT`, `ENRICHMENT.TXT`
- Q4 Web App Folder (with `Buy.TXT`, `StoreSG.db`, a template file for `server.py` and an empty `index.html` file)
- Q6 Socket Programming Folder (with `word_game.py`, `server.py`, `client.py`)

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

The number of marks is given in brackets [] at the end of each task.

The total number of marks for this paper is **100**.

Instruction to candidates:

Your program code and output for Question 1, 2, 3 and 5 should be saved in a single `.ipynb` and downloaded as

`MYE_<your class>_<your name>.ipynb`

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 1.1  
Program Code
```

Output:

In [2]:

```
#Task 1.2  
Program Code
```

Output:

In [3]:

```
#Task 1.3  
Program Code
```

Output:

- 1 A receptionist in the school manages the Lost and Found system in the General Office. Students who have lost their belongings can go to the General Office to check if it was found.

The Lost and Found system has the following menu to allow the receptionist to check, delete or add a lost item:

Menu:

1. Check for Lost and Found items
2. Delete an item returned to student
3. Add a new Lost and Found item

Select an option or press Enter to quit:

The Lost and Found items are stored as a list of strings in the system:

```
found_list = ['bottle', 'iphone', 'calculator', 'wallet', 'watch']
```

When a student goes to the General Office to look for a lost item, the receptionist will select Option 1 and enter a keyword to search for the item in the `found_list`. If the item is in the list, it will be returned to the student and she will select Option 2 to delete this item from the list. When a new lost item is brought to the General Office, the receptionist will select Option 3 to add a new keyword into the `found_list`.

Task 1.1

For Option 1, write program code for the function `check()` to:

- prompt the receptionist to enter a keyword for the lost item.
- check if the keyword exists in the `found_list`.
- display a message to indicate if the lost item has been found.

The display outputs could be as follows:

```
Please type the keyword to check: iphone
The item exists in the records.
```

```
Please type the keyword to check: bag
The item does not exist in the records.
```

[3]

Task 1.2

For Option 2, write program code for the function `delete()` to:

- prompt the receptionist to enter a keyword to delete an item
- delete the item in the `found_list` using the keyword
- display a message to indicate if the item has been deleted

The display outputs could be as follows:

```
Please type the keyword for the item to be deleted: iphone
The item has been deleted successfully.
```

```
Please type the keyword for the item to be deleted: bag
The item does not exist in the records. [3]
```

Task 1.3

For Option 3, write program code for the function `add()` to:

- prompt the receptionist to enter a keyword for the new lost item brought to the General Office.
- display the list of all the items in the `found_list`.

(You may assume that the receptionist will only enter keywords that does not exist in the `found_list` and it is not necessary to perform any input validation.)

The display outputs after deleting 'iphone' and adding 'laptop' could be as follows:

```
Please type the keyword for the item to be added: laptop
The item has been added successfully.
```

```
The list of items are: ['bottle', 'calculator', 'wallet'
'watch', 'laptop'] [3]
```

- 2 A software programmer is developing an application for a tour agency that is required to access a list of country names frequently. The programmer decides to store all the country names in a hash table to reduce the access time when performing a search on the list of country names.

Task 2.1

Write program code for the hash function `hashkey(name)` to return an integer hash address for the country name. The algorithm for calculating the hash address is as follows:

- The ASCII value for each lowercase character in the country name is computed.
- The total of these ASCII values is calculated.
- The total is divided by 30 and the remainder is returned as the hash address for the country name.

For example, `hashkey('Brazil')` will return 14 as the hash address.

[3]

The hash table `HashTable` is implemented using an array of size 30 to store all the country names.

Task 2.2

Write program code to perform the following:

- read all the country names from `COUNTRY1.TXT`
- use the function `hashkey(name)` from Task 2.1 to calculate the hash address for each country name
- store the country name in the hash table array using the hash address as the index
- if the array at a particular index is not empty, output an error message for that country.

For example, if the hash table at index 14 is not empty, the output error message could be `"Collision: Brazil is not added."`.

[7]

The following algorithm, known as Linear Probing, is one of the strategies to overcome the problem of collisions when populating a hash table:

- If the hash table array is not empty at index position i , check the next index $(i+1)$, $(i+2)$, ... until an empty position is found.
- When the last position in the hash table is not empty, start checking from the beginning of the hash table until an empty position is found.
- Insert the country name into the hash table at the empty position.

Task 2.3

Modify the program code written in Task 2.2 to manage the collisions using the Linear Probing algorithm. Read all the country names from `COUNTRY2.TXT` and store them in the same hash table, `HashTable`, used in Task 2.2. [5]

Task 2.4

Write program code for the function `search(table, name)` to search for a country name in the hash table and return the index position if found; otherwise, return `False`.

```
>>> search(HashTable, 'USA')           Output: 29
>>> search(HashTable, 'Brazil')        Output: 17
>>> search(HashTable, 'Vietnam')       Output: False
```

[5]

- 3 FastVan is a logistics company providing delivery service for online retailers like Amazon and Lazada. When a customer makes an online purchase, a job request will be sent to FastVan to collect the package from the retailer and deliver it to the customer.

The software developer at FastVan uses a First-In-First-Out (FIFO) Queue data structure. Any new delivery job will be enqueued to the tail of the queue and a completed job will be dequeued from the head of the queue.

Each delivery job is implemented as an instance of the class `Package`. The class `Package` has the following attributes and methods:

Class: <code>Package</code>		
Attributes		
Identifier	Data Type	Description
<code>data</code>	<code>STRING</code>	The <code>data</code> in the <code>Package</code> object containing the delivery job details
<code>next</code>	<code>Package</code>	(Default is <code>None</code>) The <code>next</code> pointer points to the next <code>package</code> in the queue.
Methods		
<code>constructor()</code>		
<code>get_data()</code>	<code>STRING</code>	Returns the <code>data</code> of the <code>Package</code> object.
<code>get_next()</code>	<code>Package</code>	Returns the pointer pointing to the <code>Package</code> object.
<code>set_data(new_data)</code>		
<code>set_next(new_next)</code>		

The structure for the queue has the following attributes and methods:

Class: Queue		
Attributes		
root	Package	(Default is None) The root pointer points to the head of the queue.
Methods		
constructor()		
IsEmpty()		Returns True if the queue is empty; Otherwise, returns False
enqueue(new_data)		Adds a Package object with the new_data to the tail of the queue.
dequeue()		Removes the first Package object from the head of the queue and returns its data.
display()		Prints all the data in the Package objects in the queue, from the head to the tail of the queue.

Consider the following example:

```
>>> q = Queue()
>>> q.enqueue('A')
>>> q.enqueue('B')
>>> q.enqueue('C')
>>> q.display()
```

The output for executing display() on the queue is:

```
C - B - A [head]
```

You may use the partially completed program codes for `Package` class and `Queue` class provided in the `template.ipynb` file.

Task 3.1

Write program code for the following methods to complete the implementation of the `Queue` class.

- `dequeue()`
- `display()`

[7]

Task 3.2

Write program code to create an instance of the `Queue` object, named `deliver26may`, for the following delivery jobs:

```
'AMZ001', 'LAZ001', 'AMZ002', 'SHP001', 'SHP002'
```

where `'AMZ001'` is in the head and `'SHP002'` in the tail of the queue.

[2]

Customers have the option to cancel their order and the delivery job will be removed from the queue, resulting in one fewer job in the queue.

Task 3.3

Write program code for a method `cancel(data)` in class `Queue` to remove the `Package` object containing `data` in the queue. Using the queue instance created in Task 3.2 and executing the following:

```
>>> deliver26may.cancel('LAZ001')
>>> deliver26may.display()
```

will provide the output as:

```
SHP002 -> SHP001 ->AMZ002 -> AMZ001 [head]
```

[6]

- 4 A company StoreSG supplies a wide range of products to the schools in Singapore. The company uses a database to maintain the records of all the orders made by the schools.

The table descriptions for the database are as follows:

- School (SchoolCode, SchoolName, SchoolAddress)
- Product (ProductID, ProductName, UnitCost)
- Buy (BuyID, SchoolCode, ProductID, Qty, Status)

The primary and foreign keys are indicated with underline and dashed underline respectively.

The values for the `BuyID` field in the `Buy` table are auto generated integers.

The field `Status` in the `Buy` table is used to track the status of the delivery and it could be 'Pending', 'Enroute' or 'Complete'.

The database `StoreSG.db` provided contains two of the tables `School` and `Product`, which are both populated with data.

Task 4.1

Write Python program code to:

- create a table `Buy` in the database `StoreSG.db`, with the field `Qty` defined as an integer. The foreign keys must also be referenced to the primary keys in their respective tables.
- insert all the information from the data file `Buy.TXT` into the table `Buy`.

Save your program code as `Task4_1_<your name>.py`.

[5]

Task 4.2

Hilltop University, school code 7612, has previously ordered some items from the company.

Write SQL codes to:

- a) show the name, quantity and unit cost of all the products ordered;
- b) compute the total cost of all the items ordered.

Save the two SQL codes in a text file and name it as

Task4_2_<your name>.sql [5]

The company intends to develop a web application to allow the schools to view their products and submit orders online.

Task 4.3

Write a Python program and the necessary files to create a web application that enables the list of items found in the `Product` table to be displayed by `index.html` in a web browser.

For each item, the web page should include the product's:

- ID,
- name, and
- unit cost.

Save your program as

Task4_3_<your name>_server.py and Task4_3_<your name>_index.html

Run the web application and save the output of the program as

Task4_3_<your name>_output.html [5]

[For Task 4.4 and 4.5, you are not required to include any code for input validation.]

Task 4.4

Modify the code in the `index.html` created in Task 4.3 to display a form for a user to submit an order online. The form should allow the user to order an item from the product list displayed, indicate the product's ID and the quantity to order, and submit with the school code.

Save your program code as

`Task4_4_<your name>_index.html`

Run the web application and save the output of the program as

`Task4_4_<your name>_output.html`

[4]

Task 4.5

Modify the Python program code written in Task 4.3 to:

- read the information in the order form submitted by the user .
- insert the new record into the `Buy` table and set the `Status` as `'Pending'` .
- display a confirmation page on the web browser showing the product name, the quantity ordered and the total cost for the order.

Test your web application with the following form inputs:

- School Code: 1292
- Product ID: 4
- Order Quantity: 5

Save your program as

Task4_5_<your name>_server.py

Run the web application and save the output of the program as

Task4_5_<your name>_output.html

with any additional files / sub-folders as needed in a folder named

Task4_5_<your name>

[6]

[For Question 5, you are not required to include any code for input validation.]

- 5** A computing teacher decides to use a NoSQL database to track his students' enrichment activities. The students can choose to take part in any number of enrichment activities. A list of students and their respective index numbers are stored in the file `CLASS.TXT`.

The file contains comma-separated data with the following fields:

- `name` is the name of the student.
- `index` is a unique positive integer number for each student.

The data is sorted in ascending order of the names.

Task 5.1

Write Python program code to:

- create a NoSQL database `computing`, drop the database if it exists before creating it.
- insert the data from the file `CLASS.TXT` into the collection `enrichments`.

[6]

The list containing the names of students and the enrichment activities they participated in can be found in the file `ENRICHMENT.TXT`.

The file contains comma-separate data with the following fields:

- `activity` is the name of the enrichment activity.
- `index` is the index number of the student who participated in the enrichment activity.

Task 5.2

Write Python program code to:

- read each line of record in the data file `ENRICHMENT.TXT`
- use the value for the `index` field to locate the student's document in the collection created in Task 5.1
- update the student's document by adding a new field with the name of the activity as the fieldname and assign `'1'` as its value
- add additional field(s) to the students' document if they have participated in more than one enrichment activity.

For example:

The student `Andy (index:1)` has taken part in the enrichment activities `Cyberthon` and `BuildingBloCS`, and his document would be:

```
{'name':'Andy', 'index':'1', 'Cyberthon':1,  
'BuildingBloCS':1}
```

[5]

During the academic year, some students withdraw from the college and the student's document needs to be deleted from the collection to comply with the Personal Data Protection Act (PDPA).

Task 5.3

Write Python program code to:

- display the list of students, showing only the `index` and `name`.
- prompt the user to enter the `index` of the student to be deleted.
- delete the student's document from the collection `enrichments`.
- display the updated list of the remaining students, showing only the `index` and `name`.

[4]

- 6 In a word game for young children, the program will randomly choose five four-letter words from its repository, scramble each word, and let the player input the correct arrangement of the words.

An example of the word game is as follows:

```
Enter the word you can form with "ITGF":
```

A working program code of the game in `word_game.py` is provided for you to examine how the game is executed.

Task 6.1

Modify the program code for the game to track the number of correct answers and return an output at the end of the game:

```
You have answered ____ out of 5 correctly. [3]
```

Task 6.2

Using the templates, `server.py` and `client.py`, provided, write the server and client program codes for this Word game. [13]

server.py

```
from socket import socket

my_socket = socket()
my_socket.bind(('127.0.0.1', 12345))
my_socket.listen()

new_socket, addr = my_socket.accept()
print('Connected to: ' + str(addr))

# write your code below:

new_socket.close()
my_socket.close()
```

client.py

```
from socket import socket

my_socket = socket()
my_socket.connect(('127.0.0.1', 12345))

# write your code below:


my_socket.close()
```

Save the modified `word_game.py`, `server.py` and `client.py` files in the **Q6 Socket Programming** folder.

BLANK PAGE